Harness Engineering

Everything around the model that makes it an agent

Anatomy · Tools · Boundaries · Verification
Long horizons · Orchestration · Maturity · Practice

Part 1: Foundations
What a harness is · Harness anatomy

Part 2: Tools and boundaries
Tool design · Execution boundaries and guardrails

Part 3: Verification and long horizons
Verification loops · Long-horizon execution

Part 4: Orchestration and maturity
Multi-agent harnesses · Maturity and failure modes

Part 5: Practice
Repo as system of record · Session lifecycle · Scope and done

Notes: ai-engineering/03-harness-engineering

PART 1

Foundations

What a harness is · Harness anatomy

The thesis

The expanded equation, on one refund

Why the harness dominates the model

Same model, better interfaces

Four nested scopes of engineering

Where context ends and the harness begins

Failures are configuration, not model limits

The four parts every harness needs

The ratchet

Five subsystems, one per failure

One small harness, all five subsystems

The minimum viable harness

The thesis

Agent = Model + Harness.

Everything between the request and the output, except the weights, is harness. A decent model with a great harness beats a great model with a bad harness.

The expanded equation, on one refund

Agent = LLM + [Context + Tools + Constrain + Verify + Correct] = Model + Harness

Function	In the refund example	Design principle
Context	The 7-day refund policy is in the system prompt	Information sufficiency at every decision
Tools	`query_order`, `process_refund`	Clear interface: names, examples, boundaries
Constrain	Refund ≤ order total, checked by the framework	Fail-safe defaults: off until enabled
Verify	Confirm in the DB that the refund landed	Check structured fields, never model prose
Correct	Retry automatically on API timeout	Hide intermediate state until unrecoverable

> **Context + tools is the demo. Constrain, verify and correct are the production harness.**

Source: notes/01-what-a-harness-is.md

Why the harness dominates the model

Reliability compounds

Steps compound

A 90%-reliable step chained five times lands near **59%**. Real tasks run far more than five steps once you count tool calls and parses.

Harness-only gains

LangChain DeepAgents: **52.8%** → **66.5%** on Terminal Bench 2.0, rank ~30 → top five, on a fixed model.

Same model, two harnesses

Opus 4.6 scores far below its ceiling in a loose harness and far above it with tighter tools and prompts.

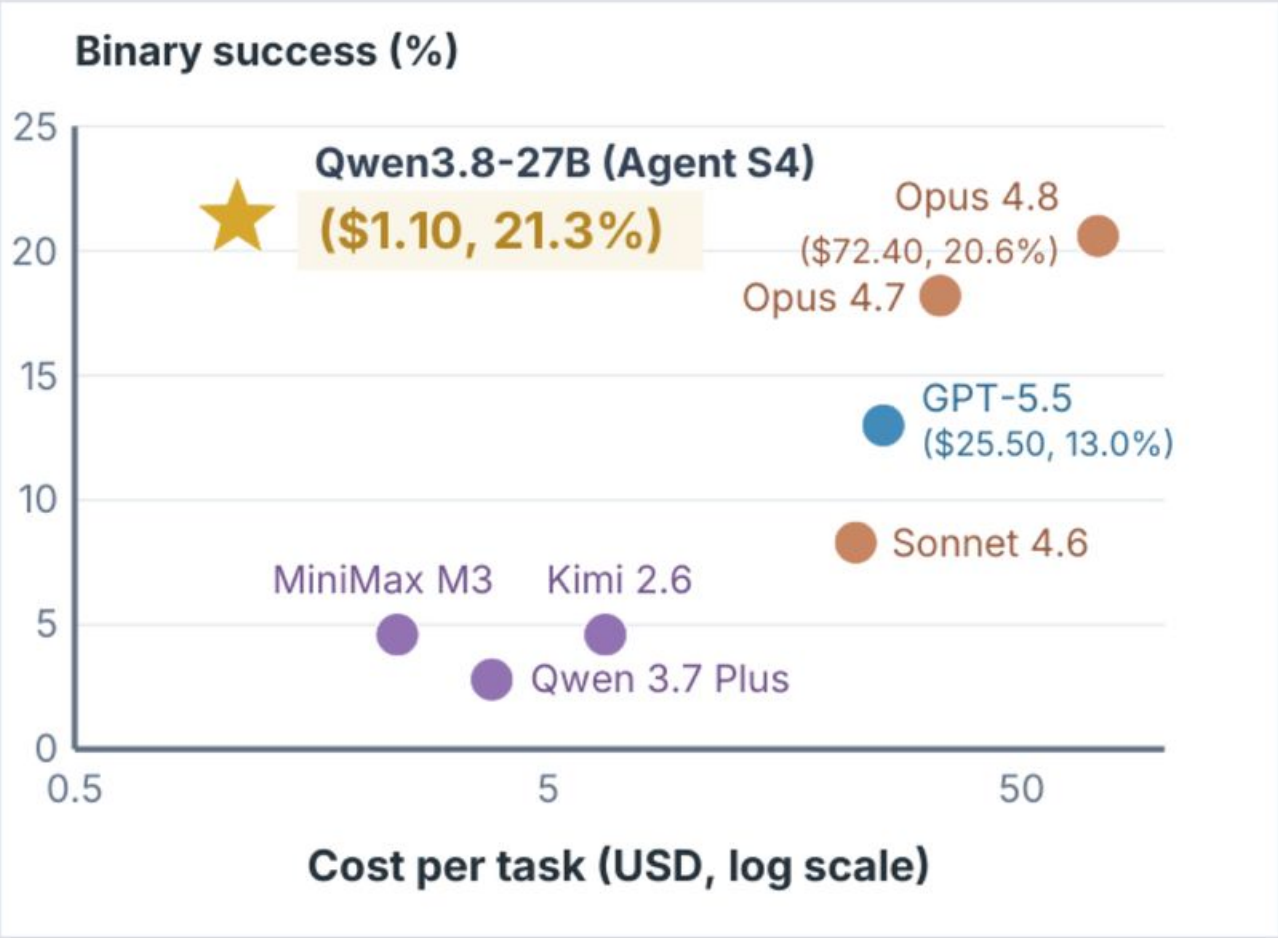
So…

When an agent fails, audit orchestration first, then context. In a mature setup the model is rarely the bottleneck.

Source: notes/01-what-a-harness-is.md

Same model, better interfaces

Agashe et al., Agent S4



OSWorld V2 binary success against cost per task (Agashe et al., 2026)

- Only the **interfaces** evolved: tools, observation formats, when-to-use notes
- Qwen3.8-27B: **5.6%** → **21.3%** (3.8×)
- Above Opus 4.8 (20.6%) and GPT-5.5 (13.0%)
- At about **1.5%** and **4.3%** of their cost per task

Source: notes/01-what-a-harness-is.md

Four nested scopes of engineering

prompt ⊂ context ⊂ harness ⊂ loop

wider scope

1. **Prompt** — The instruction. Claimed reliability impact **5–15%**
2. **Context** — The working set on one call. Claimed impact **15–30%**
3. **Harness** — The system the model runs in. Claimed impact **50–80%**
4. **Loop** — Sustained operation across runs (pillar 04)

The percentages are directional, not measured (harness-engineering.ai). The ordering is the point.

Source: notes/01-what-a-harness-is.md

Where context ends and the harness begins

Context engineering	Harness engineering
What goes in the window on **this** call	The loop that decides **whether and when** to call
Which documents, memory, tool results to inject	Which tools exist at all, and what they may do
Token budget within one assembly	Cost envelope across a whole run
Compaction as a technique	Compaction as middleware that fires on a trigger

> **Once assembly is stateful and conditional, you have crossed from context into harness.**

Source: notes/01-what-a-harness-is.md

Failures are configuration, not model limits

HumanLayer's "skill issue"

Failure	Not this	But this
Ignored a convention	"The model is dumb"	The convention isn't in `AGENTS.md`
Ran `rm -rf`	"The model is unsafe"	There's no blocking hook
Lost the thread at step 40	"Window too small"	No planner/executor split, no plan file
Shipped broken code	"It can't code"	No typecheck back-pressure in the loop
Rated its own work 9/10	"It's sycophantic"	Generator and evaluator are the same agent

> **Every right-hand cell is a build task. That is the whole discipline in one table.**

Source: notes/01-what-a-harness-is.md

The four parts every harness needs

The minimal decomposition

Acceptance baseline

What "done and correct" means, checkably. The most often **faked** part.

Execution boundary

Where the agent runs and what it can touch.

Feedback signals

How the agent learns its last action was wrong.

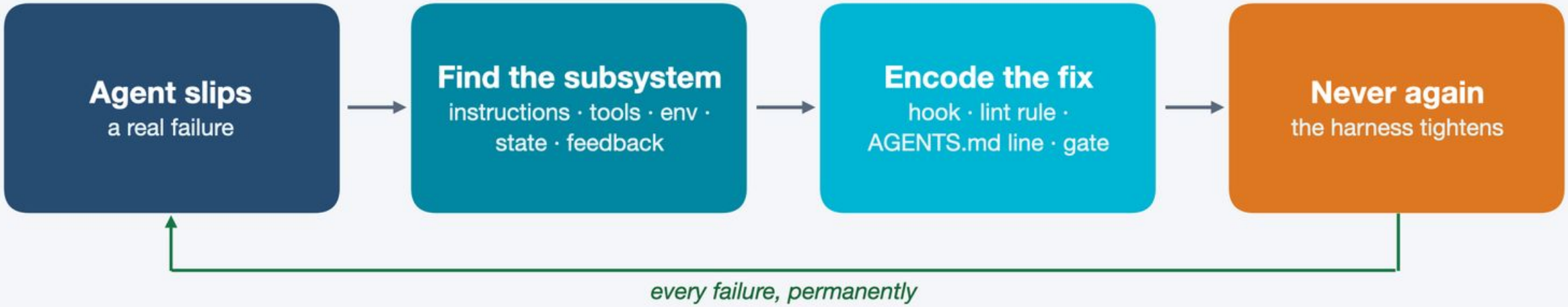
Rollback

How a bad action gets undone. The most often **missing** part.

Missing any one, it is not a harness. It is a prompt with tools attached.

Source: notes/01-what-a-harness-is.md

The ratchet



- Every line in a good `AGENTS.md` traces back to something that went wrong
- Keep `AGENTS.md` under **~60 lines**: a pilot's checklist, not a style guide
- Lines that can't name their originating failure are speculative; delete them

Source: notes/01-what-a-harness-is.md

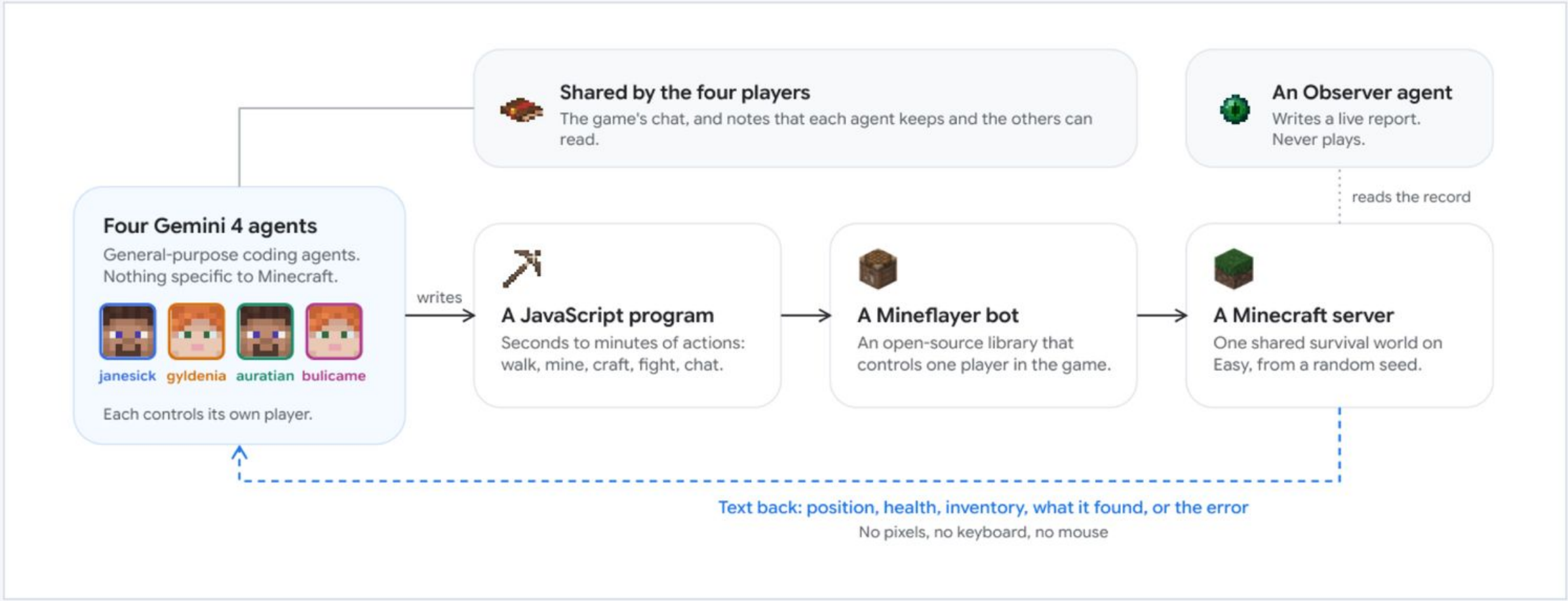
Five subsystems, one per failure

Subsystem	Question it answers	Components
Instructions	What should I do, by what rules?	System prompt / `AGENTS.md`, memory
Tools	What can I act with?	Tools, skills, MCPs, bash, context tools
Environment	Where do I run, and does it start?	Filesystem, sandbox, `init.sh`
State	What has happened so far?	Plan file, progress log, feature list, git
Feedback	Did it work?	Hooks, test runners, verification gates, traces

> **Agents are rarely stuck. They are confidently wrong, which is a feedback gap.**

Source: notes/02-harness-anatomy.md

One small harness, all five subsystems



Four coding agents, one shared server, a fifth Observer that never plays (Faldor et al., 2026)

Source: notes/02-harness-anatomy.md

The minimum viable harness

Six items

1. **Tool loop** — with typed schemas
2. **Sandbox** — even just a working-directory boundary
3. **Filesystem** — the agent can read and write it; the most foundational primitive
4. **Plan file** — for anything past ~5 steps
5. **Verification gate** — one binary pass/fail check
6. **Structured traces** — of every step

> **Build capability first, then the constraints that make it survive scale, then productize.**

Source: notes/02-harness-anatomy.md

PART 2

Tools and boundaries

Tool design · Execution boundaries and guardrails

- Design for an agent, not a programmer
- The five-section tool contract
- Four questions before a tool ships
- Discover tools; don't preload them
- A skill description is routing logic
- Coding agents edit by exact match
- Requests vs. rules
- The isolation ladder
- Silent success, verbose failure
- Test the gate, not the model's mood
- Five protection layers

Design for an agent, not a programmer

ACI, not API

An API's reader

- Has docs, type hints and autocomplete
- Can ask a follow-up question
- Learns boundaries by reading the codebase

Implicit knowledge fills the gaps

An ACI's reader

- Sees only a name, a description and a schema
- Needs explicit boundaries between tools
- Needs error-proof params: enums, absolute paths
- Needs examples **inside** the definition

What it can't see doesn't exist

When a tool is misused, check the description before questioning the model.

Source: notes/03-tool-design.md

The five-section tool contract

Section	Purpose
What it does	The operation, plainly
WHEN TO USE	Positive triggers the tool owns
WHEN NOT TO USE	Boundary cases that belong to a neighbouring tool
DO NOT USE FOR	Hard negatives: misuse seen in practice (ratchet output)
EXAMPLES	Concrete invocations with realistic arguments

> **Adding 1–5 examples moved tool-call accuracy from ~72% to 90% (Li).**

Source: notes/03-tool-design.md

Four questions before a tool ships

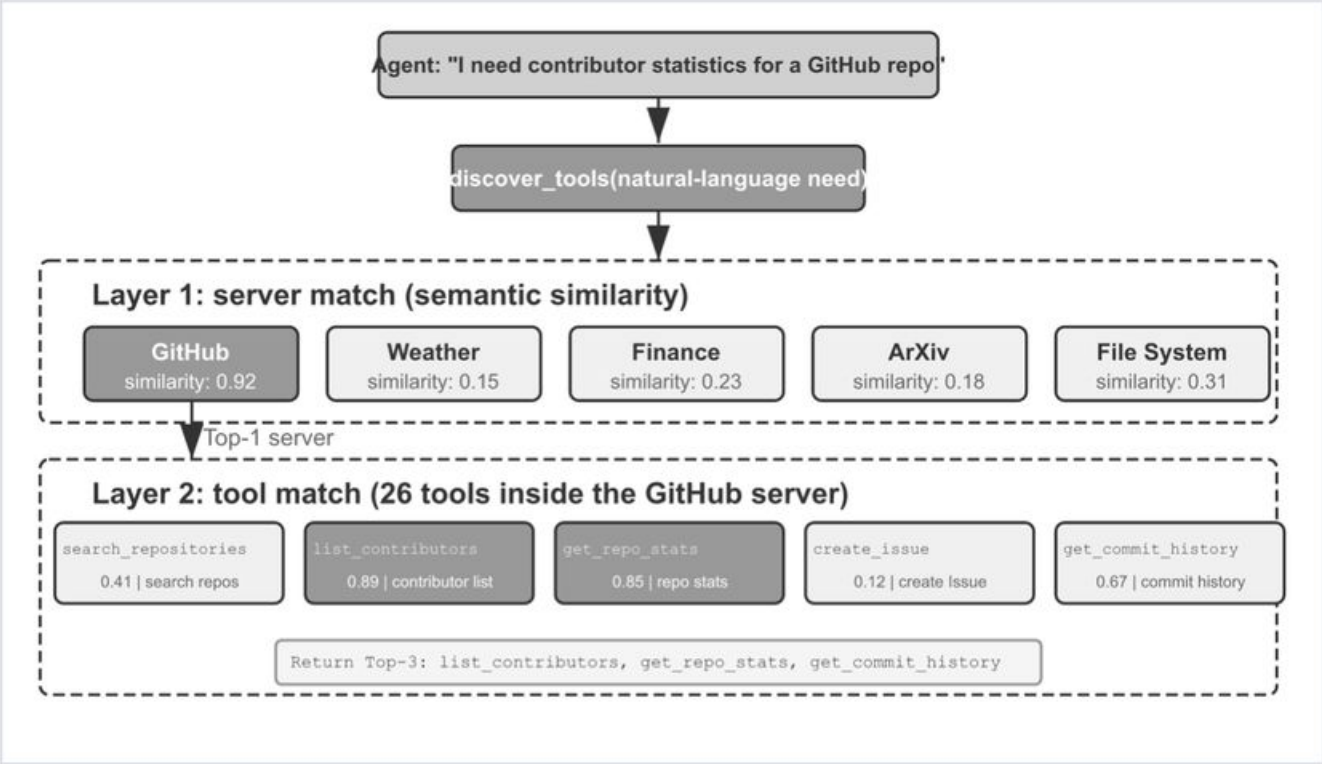
Question	If the answer is no
Reversible?	Put a confirmation gate in front of it
Idempotent?	Never retry it without an idempotency key
Observable?	It can't be debugged; add a trace
Parallel-safe?	Serialize it in the orchestrator

> **Gate the irreversible. Gating reversible actions is the fastest way to make an agent useless.**

Source: notes/03-tool-design.md

Discover tools; don't preload them

Li, Fig. 4-2



Two-level matching, server then tool (Li, AI Agents in Depth, 2026, Apache-2.0)

- Five MCP servers ≈ **55K tokens**, ~30% of a 200K window, before the first message
- MCP-Zero: **~98% fewer tokens** across ~2,800 tools
- Dynamic discovery: Opus 4 **49% → 74%** on tool selection
- Cursor's greppable tool folder: **–46.9%** tokens
- Append new schemas at the end and leave them pinned, so the KV cache survives

Source: notes/03-tool-design.md

A skill description is routing logic

YAML

```yaml
# bad (~45 tokens): details that belong in the body
description: |
 Handles the complete deployment process to
 production: environment checks, rollback,
 post-deploy verification. Use before deploying.

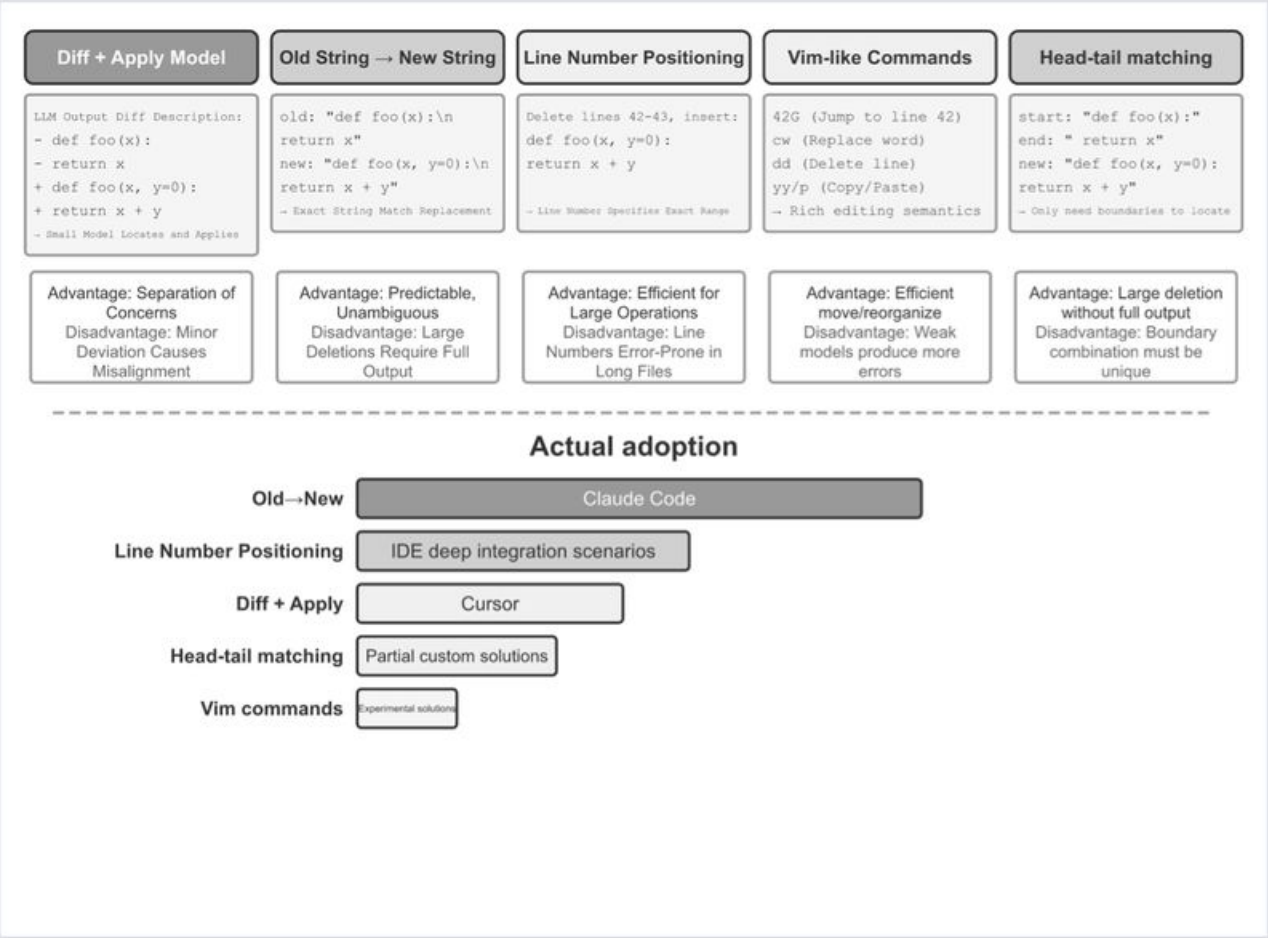
# good (~9 tokens): the same routing signal
description: Use when deploying to production
 or rolling back.
```

- Only name + description stay resident; the body loads on demand
- Add **negative examples**: when not to fire, and what to use instead
- Templates and worked examples go in the body, where they are free until used
- For determinism, invoke explicitly: `Use the <name> skill.`
- Skills + shell allowed; network on a **minimal allowlist**

Source: notes/03-tool-design.md

Coding agents edit by exact match

Li, Fig. 5-4



Five file-editing schemes and their trade-offs (Li, AI Agents in Depth, 2026, Apache-2.0)

- **Old → new string** (Claude Code): predictable; fails loudly if not unique
- **Start + end anchors:** same reliability, cheaper large deletions
- **Line ranges:** models miscount, and numbers shift after every edit
- **Diff + apply model** (Cursor): fast, needs a trained apply model
- Search: Claude Code greps with no index; Cursor pays for one

Source: notes/03-tool-design.md

Requests vs. rules

A `CLAUDE.md` line is a request the model can ignore. A pre-tool-use hook is a rule it cannot.

Bucket	Enforcement	Example
Always do	Instruction in `AGENTS.md`	Run the formatter before committing
Ask first	Permission gate or HITL confirmation	Add a dependency, change a schema
Never do	Hook or permission deny, never a sentence	Write `.env`, force-push, delete outside the workspace

> **Match the enforcement level to the cost of getting it wrong.**

Source: notes/04-execution-boundaries.md

The isolation ladder

stronger isolation ↓

× **venv** — Not on the ladder: isolates dependencies, not effects

1 **OS-level** — macOS Seatbelt, Linux seccomp / Landlock

2 **Container** — Process, filesystem and network namespaces

3 **microVM** — Firecracker-class; for untrusted code and production agents

Pair every rung with quotas: CPU, memory, disk, network, wall-clock.

Source: notes/04-execution-boundaries.md

Silent success, verbose failure

Hook that announces every pass

- Burns tokens on every edit
- Trains the model to skim hook output
- The one real failure is lost in noise

Low signal

Hook that speaks only on failure

- Full error, file and line
- The agent learns within **one step**
- Typecheck and lint become back-pressure

High-signal channel

Hooks run as code on a trigger, after every edit and before every risky command.

Source: notes/04-execution-boundaries.md

Test the gate, not the model's mood

Run each deletion route twice: baseline (canary must die) and guarded (must survive **and** log a denial).

Route	Verdict with a `Bash(rm:*)` deny rule
`rm -f`	**HELD**: blocked as intended
`find -delete`	**INCONCLUSIVE**: bypassed, no refusal recorded
File-writing tool overwrite	**BYPASSED**: canary destroyed

> **Enumerate effects, not commands. `rm` is one spelling of delete; prefer allowlists and real sandboxes.**

Source: notes/04-execution-boundaries.md

Five protection layers

Defence in depth

Pre-input

Malicious or malformed user input

Pre-retrieval

Poisoned documents entering context

Pre-generate

Unsafe assembled context reaching the model

Post-generate

Unsafe output leaving the system

Escalation

Anything unresolved reaching a human

So…

Tool inputs are model-written, so they are **semi-trusted**. Validate them against the schema like an external request.

Each guardrail lives in the component it protects, not in one central service.

Source: notes/04-execution-boundaries.md

PART 3

Verification and long horizons

Verification loops · Long-horizon execution

Three layers before "done"

Spend at the gate, and only where it decides

Separate generation from evaluation

An observer checks claims against the record

Sprint contracts are written before the work

The verification ladder, cheapest first

Not every failure is retryable

Every recovery path needs a ceiling

Make the loop re-entrant

What makes an agent deep

Compaction vs. context reset

The handoff separates verified from assumed

Ralph loops defeat context anxiety

What a 27-hour run looks like

Three layers before "done"

Layer	Checks	Proves
1. Static	Syntax, types, lint, architecture rules	It's well-formed
2. Runtime	Tests run, app reaches ready, side effects correct	It runs
3. System	End to end as a user drives it, real dependencies	It's **correct**

> **Don't run level N+1 while level N fails. Skipping a required level means not done.**

Source: notes/05-verification-loops.md

Spend at the gate, and only where it decides

Make the verifier smarter

- Cheap fast model generates; strong model checks at the gate
- Same model checking itself gives **confident agreement**
- Use a **second librarian**: another model family

Verification runs once per gate

Delete a verifier that cries wolf

- PRINCE (Bayer) **removed** its LLM SQL reviewer: false positives
- Kept the exact checks: `SELECT`-only allowlist, result cap
- A gate people route around is worse than none

Cheapest mechanism that is decisive

A verifier earns its place only if acting on its verdict beats ignoring it.

Source: notes/05-verification-loops.md

Separate generation from evaluation

Anthropic, long-running apps

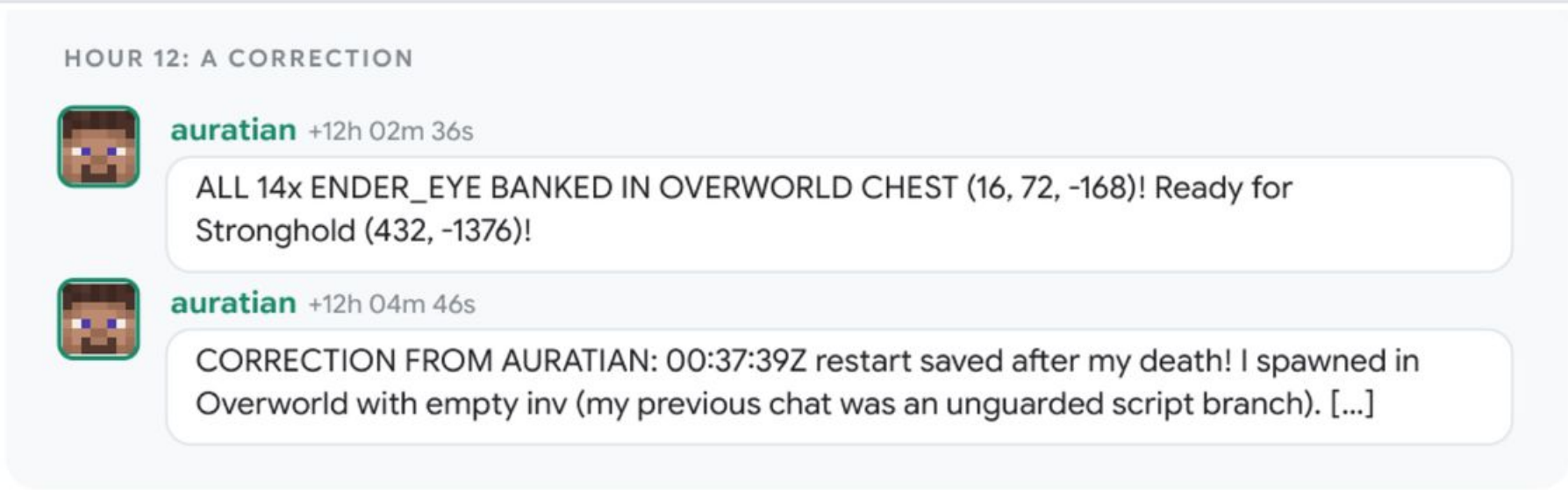


- Solo run: **20 min, $9**, core gameplay broken
- Full harness: **6 hr, $200**, a working game
- Browser DAW: **$124.70** total; QA only **$10**, and it caught presentational-only features
- The evaluator needs calibrating too, with hard per-dimension thresholds

Source: notes/05-verification-loops.md

An observer checks claims against the record

Faldor et al., hour 12



A claim of 14 banked Ender Eyes, retracted two minutes later (Faldor et al., 2026)

Source: notes/05-verification-loops.md

Sprint contracts are written before the work

MARKDOWN

```
# Sprint contract: dark mode
## Scope
theme toggle · CSS variables · dark-mode tests
## Verification
visual regression per component · main-flow e2e
no flash of unstyled content
## Exclusions
print styles · third-party component theming
```

- Both sides negotiate it in writing, before code
- The evaluator can't invent criteria afterwards
- The generator can't redefine done
- **Exclusions** is the section people skip

> **Name the dimensions, weight them, score each separately. One 1–10 score gives nothing to act on.**

The verification ladder, cheapest first

costlier, more nuanced ↓

1. **Schema check** — Does it parse and conform?
2. **Reference comparison** — Does it match known-good output?
3. **Calibrated LLM judge** — Rubric-scored, and it returns **why** it failed so the agent can revise
4. **Human stop** — For what neither code nor a judge can decide

> **Most production systems run at least two. Pass@k shows capability; ship on Pass^k.**

Source: notes/05-verification-loops.md

Not every failure is retryable

Failure	Response
Transient (network, rate limit)	Backoff 2×, jitter ±20%, max 3
Tool error	Retry once, then escalate
Model refusal	**Never retry**: it will refuse again
Schema violation	Retry once with the validation error in context
Context exhaustion	Compact or reset, then resume
Fatal	Stop and surface the error

> **Budget retries per invocation, not per tool. Degrade loudly with a `degraded` flag.**

Source: notes/05-verification-loops.md

Every recovery path needs a ceiling

Two-retry rule

A third failure on the same gate means the context or the spec is wrong, not the execution.

Loop detection

Count edits per file. After N, prompt "reconsider your approach": force a re-plan, not a repair.

Feed the error back

Give the agent the error, the query and the context. PRINCE caps it at **3** attempts.

Set ceilings from data

Before Claude Code's breaker (trips at **3**), one session failed **3,000+** times; futile retries cost **~250K** API calls a day.

Source: notes/05-verification-loops.md

Make the loop re-entrant

Progress lives in files, not in the context window.

Once state is external, any fresh context can pick up where the last one stopped. Context windows become disposable.

What makes an agent deep

A shallow agent runs one tool loop in one window. A deep agent adds four things.

Pillar	What it means	Deep Agents ships
Explicit planning	A living to-do list the agent updates	`write_todos`
Orchestrator + subagents	Specialists with clean contexts	`task` tool
External memory	Work goes to files, not history	Virtual filesystem, `AGENTS.md`
Verification	Output checked before it's accepted	HITL `interrupt_on`

> **The agent loop is the same everywhere. The defaults are the harness.**

Source: notes/06-long-horizon-execution.md

Compaction vs. context reset

	Compaction	Context reset
Mechanism	Summarize earlier turns in place	Clear the window, hand off via files
Preserves	A lossy trace of everything	Only what the handoff captures
Fails by	Summary-of-summary drift	Anything not written down is gone
Best for	One long session	Multi-day work

Resets beat compaction on Sonnet 4.5; later models need them less. Re-test on every upgrade.

Source: notes/06-long-horizon-execution.md

The handoff separates verified from assumed

Session handoff skeleton

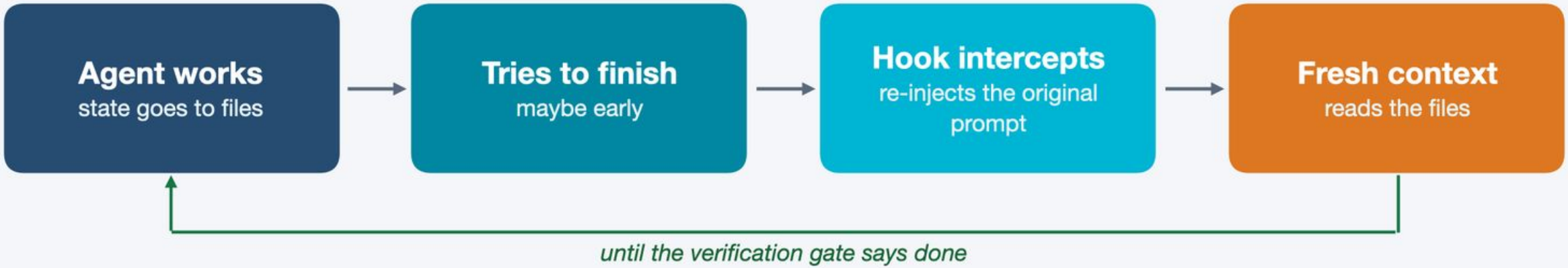
MARKDOWN

```
## Verified Now
what works · what verification actually ran
## Changed This Session
code or behaviour added · harness changes
## Broken Or Unverified
known defect · unverified path · risk
## Next Best Step
feature · why next · what counts as passing
## Commands
startup · verification · focused debug
```

- Start writing it at **~60%** of the window, while there is room
- A generated summary blurs verified and assumed; this doesn't
- Pair with `progress.md`: current state on top, append-only log below
- Exit only on a clean state: build, tests, logs, `./init.sh`

Source: notes/06-long-horizon-execution.md

Ralph loops defeat context anxiety



Premature wrap-up becomes harmless, because "done" opens a new window instead of ending the work.

What a 27-hour run looks like

Faldor et al., 27 h 14 min



Four agents, 27 h 14 min, 605 deaths along the bottom (Faldor et al., 2026)

Source: notes/06-long-horizon-execution.md

PART 4

Orchestration and maturity

Multi-agent harnesses · Maturity and failure modes

- Subagents are a context firewall first
- Protocols precede collaboration
- Four multi-agent topologies
- Multi-agent failures are harness bugs
- The five-stage maturity model
- The six-stage task pipeline
- Iterative simplification
- Eight deployment anti-patterns

Subagents are a context firewall first

- A search that reads **40** files and finds **3** returns the 3
- The orchestrator gets a **~1,000–2,000-token** summary, not the exploration
- Orchestrator holds the plan; subagents hold the details
- Deep Agents' `task` tool: fresh instance, runs to completion, **one** final report
- Isolation is the most expensive context lever; use it at a real boundary

The sizing rule

Give each subagent the minimum context, authority and execution budget needed to finish one clearly defined task.

Source: notes/07-orchestration.md

Protocols precede collaboration

Foundation	Defines	Prevents
Communication protocol	Message meanings; requests, results, failures, acks	Lost acks, silent failures, double runs
Task graph	What work exists, who owns it, what depends on what	Duplicate owners, premature starts
Isolation boundary	Which files, branches, tools each agent may touch	Write collisions

> **Establish the task graph and isolation boundaries before you add parallelism.**

Source: notes/07-orchestration.md

Four multi-agent topologies

Pick the shape, then the cut

Sequential

A → B → C. Stages that genuinely depend on each other.

Parallel

Fan out, join. Subtasks that share no state.

Manager hierarchy

One orchestrator delegates and integrates. The default for open-ended work.

All-to-all

Free chat. Brainstorming only: hard to predict or control.

Cut the work by function, by file (spatial), by stage (temporal) or by data partition.

Source: notes/07-orchestration.md

Multi-agent failures are harness bugs

Named failure modes

Failure	Shape	Mitigation
Loop oscillation	Two nodes bounce work forever	Counters on cyclic edges; escalate
Context pollution	Nested flows leak state upward	Boundary schemas; return summaries
Concurrency collision	Branches write the same state	JoinNode barriers; key ownership
Consensus deadlock	Reviewers never agree	Tie-break, majority or an arbiter

> **E2E fails but components pass → a handoff bug. Evaluate both.**

Source: notes/07-orchestration.md

The five-stage maturity model

Stage	Name	What exists
0	Ad-hoc	Scripts, manual tool calls, no registry or structured logs
1	Basic	Schema-first tools, a registry, unit-test verification
2	Verified	CI checks, sandboxing, tracing, behavioural evals, branch-per-agent
3	Observability-first	E2E tracing, LLM-judge scoring, middleware, versioned memory
4	Self-healing	Auto-remediation, cost-aware orchestration (emerging, 2026)

> **Stage 1 makes it work, 2 makes it trustworthy, 3 makes it improvable. Most claiming 3 are at 1.**

Source: notes/08-maturity-and-failure-modes.md

The six-stage task pipeline

Per task, in a mature harness

1. **Preflight** — Gather context; establish current state
2. **Plan** — Decompose into steps
3. ★ **Approve** — Mandatory human gate on the **plan**
4. **Tasks** — Execute, documenting as you go
5. **Verify** — Binary pass/fail quality gate
6. **Finish** — Export artifacts, log the outcome

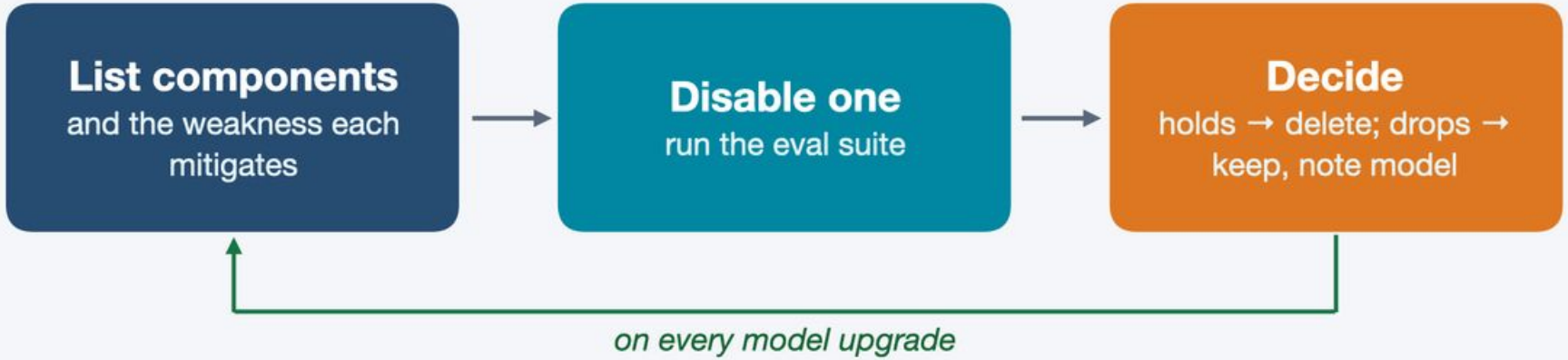
> **Reviewing a wrong plan takes two minutes. Reviewing the code from a wrong plan takes an hour.**

Source: notes/08-maturity-and-failure-modes.md

Iterative simplification

The counterweight to the ratchet

Every component encodes an assumption about what the model can't do on its own.



- Opus 4.6 let Anthropic **remove sprint decomposition** with no quality loss
- PRINCE deleted a verifier that never paid off, with no upgrade needed
- Scaffolding for a model weakness expires; scaffolding for an external constraint doesn't

Source: notes/08-maturity-and-failure-modes.md

Eight deployment anti-patterns

Looks like the model; is the engineering

Anti-pattern	Fix
System prompt as knowledge base	Conventions in the prompt; knowledge in skills
Unbounded tool count	Merge overlaps, namespace the rest
No verification loop	Acceptance criteria per task type
Boundaryless multi-agent	Roles, permissions, a worktree per agent
Unmanaged memory	Watch tokens; compact or reset at a threshold
No evaluation	Turn every failure into a test case
Premature multi-agent	Find the single-agent limit first
Constraints by expectation	Enforce with tools, linters, hooks

> **The five big failure modes sit above these. Skipping the harness hides the other four.**

Source: notes/08-maturity-and-failure-modes.md

PART 5

Practice

Repo as system of record · Session lifecycle · Scope and done

The fresh-session test

AGENTS.md is a router, not a manual

One feature at a time, passed by evidence

The fresh-session test

Open a new session with no chat history and ask five questions.

Question	Answered by
What is this system?	`AGENTS.md` / README
How is it organized?	`ARCHITECTURE.md`, module docs
How do I run it?	`init.sh`, Makefile, package scripts
How do I verify it?	Test, lint and check commands
Where are we now?	Progress log, feature list, git history

> **Every unanswered question gets guessed, and every new session guesses again.**

Source: notes/09-repo-as-system-of-record.md

AGENTS.md is a router, not a manual

Progressive disclosure

MARKDOWN

```
# AGENTS.md
## Overview
Python 3.11 FastAPI backend, PostgreSQL 15.
## Quick start
Install: make setup · Test: make test
## Hard constraints      (top, ≤15, non-negotiable)
- All DB queries use parameterized SQLAlchemy 2.0
- All PRs pass pytest + mypy --strict + ruff
## Topic docs            (what + when to read)
- docs/api-patterns.md: when adding endpoints
- docs/database-rules.md: when touching DB code
```

- Root file: start near **60 lines**; topic docs 50–150
- Red lines at the top or bottom, never the middle (lost in the middle)
- Each rule records its source, scope and expiry
- Audit and delete dead rules like unused dependencies

Source: notes/09-repo-as-system-of-record.md

One feature at a time, passed by evidence



`blocked` hangs off active. A script flips the state; the agent requests verification, never edits the field.

Verified completion rate = verified ÷ activated. Under WIP = 1 it sits at 1.0 between features.

Source: notes/10-scope-and-done.md

Rules of thumb

The pillar on one slide

Wrap-up

Diagnose

Failures are configuration. Fix the harness before blaming the model, and ratchet every fix in.

Tools

Write descriptions for a reader with no docs. Fewer, clearer tools. Gate only the irreversible.

Boundaries

Must hold every time → hook, permission or schema. Test the gate; enumerate effects, not commands.

Verification

Binary evals first. A separate, smarter evaluator. Two retries, then fix the spec.

Long horizons

State in files. Handoffs that separate verified from assumed. Clean state is part of done.

Maturity

Approve the plan. Fewest agents the task allows. Strip scaffolding on every model upgrade.

Source: notes/08-maturity-and-failure-modes.md

Where to go next

Wrap-up

- **04 — Loop engineering:** the loops the harness runs, and how they improve themselves
- **06 — Eval:** graders, calibration and pass^k, the binary evals every rule here depends on
- **Read:** Anthropic on long-running harnesses; OpenAI on harness engineering; Li, AI Agents in Depth

The one idea to keep

The model holds the intelligence; the harness makes it useful. When an agent fails, change the system around it, then make sure that failure can never happen again.

Source: README.md